Algebraic World Model: From Observations to Axioms and Proofs
Supplementary Material

Anonymous authors
Paper under double-blind review

References to sections, figures, tables and equations without a letter refer to the main paper.

A Formalization details

A.1 Environment. Following Zhao et al. (2026), an environment is a deterministic, goal-directed special case of a partially observable Markov decision process (Kaelbling et al., 1998): states $\mathcal{S}$, actions $\mathcal{A}$, observations $\mathcal{O}$, a transition function $T : \mathcal{S} \times \mathcal{A} \to \mathcal{S}$, an observation function $\rho : \mathcal{S} \to \mathcal{O}$ and goal states $G \subseteq \mathcal{S}$. The agent sees $o_t = \rho(s_t)$ and receives a signal when a level is cleared. A transition is a triple $x = (o, a, o')$, and a game's transition corpus $\mathcal{D}$ is the set of distinct transitions across its runs, deduplicated by content hash.

A.2 Concepts and the gate. A concept $c = (\sigma_c, \alpha_c, \gamma_c, E_c)$ has an optional executable scope σ_c, an assertion α_c returning TRUE, FALSE or NA, an optional predictor γ_c, and a non-empty evidence set E_c. When the scope rejects x the verdict $v_c(x)$ is NA, otherwise it is $\alpha_c(x)$; an exception, an invalid return value or a timeout gives ERR. A concept is scored only on transitions outside E_c; a concept with no TRUE or FALSE verdict there is reported as unmeasured, not blocked. The gate of Eq. (1) is enforced on the action command: it refuses the next action while the latest settlement has a FALSE or ERR verdict on any $x \in \mathcal{D}$, including the concept's own evidence.

A.3 Objects, rules and unification. A partial parser $\pi : \mathcal{O} \rightharpoonup \mathcal{W}$ maps a screen to a typed world whose fields are the object types; it is the only code of the rewritten model that reads pixels, and a screen it cannot parse is reported rather than guessed. A rule $r = (\sigma_r, \alpha_r)$ has a scope and an assertion on $(\pi(o), a, \pi(o'))$. Each rule is scored on every recorded step by the number of agreeing and disagreeing steps; a step on which no rule gives a verdict is unclaimed. The corrupted transitions $\mathcal{N}$ in (**??**) number at most 120 per game, with a fixed seed; each replaces part of a recorded transition (o, a, o') in one of five ways: the successor is o itself (nothing happened); the successor of the same action on another transition; the predecessor of another transition; o' with a few cells changed that do vary in the corpus; or another transition's o (swapped precondition). We set $\mathrm{Rej}(r) = \{m \in \mathcal{N} : v_r(m) = \mathsf{FALSE}\}$. A field is eliminated when erasing it to a constant changes no verdict on $\mathcal{D} \cup \mathcal{N}$.

A.4 Lean statements. A step is $\mathrm{step} : \mathcal{W} \times \mathcal{A} \to \{\mathrm{ok}(w'), \mathrm{won}(w')\}$, defined by dispatching among the rule definitions. $\mathrm{everWon}(w, \bar{a})$ holds when some prefix of $\bar{a}$ ends in won. The two statements of §2.4 are, in Lean,

$$\mathrm{Winnable}(w) :\Leftrightarrow \exists\, \bar{a}.\ \mathrm{everWon}(w, \bar{a}) = \mathsf{true}, \qquad \mathrm{Unsolvable}(w) :\Leftrightarrow \forall\, \bar{a}.\ \mathrm{everWon}(w, \bar{a}) = \mathsf{false}.$$

For level ℓ, the initial world w_0^ℓ is parsed from that level's first screen in the certified run (for some games, hidden fields read from another frame of the same level are supplied), and the certified actions are $\bar{a}^\ell$; the reachability proof (a replay of a recorded run; see §3 on what this measures) establishes $\mathrm{traceOk}(w_0^\ell, \bar{a}^\ell)$, i.e. that every computed state agrees with the recorded frame on the components the formal model represents, and establishes $\mathrm{Winnable}(w_0^\ell)$ with $\bar{a}^\ell$ as witness. Each theorem is closed by kernel reduction and its axioms lie within $\{\texttt{propext}, \texttt{Classical.choice}, \texttt{Quot.sound}\}$, with no `native_decide`.

A.5 The direction of the guarantee. Let $h : \mathcal{S} \to \mathcal{W}$ be the abstraction realized by the parser and V the set of visited state–action pairs. Agreement with the record gives: for $(s, a) \in V$, $\mathrm{step}(h(s), a)$ agrees with $h(T(s, a))$, and nothing outside V. If the model over-approximates the environment

Table 1: The formal side of each game's model. Level count and certified action count come from the ARC Prize scorecard; the remaining columns are computed from the repository. "Rules (Lean)" counts the rules translated into Lean. "Steps explained" is counted over the whole record, including exploration, so it can exceed the certified action count; for most games it counts the steps on which the Lean model's replay agrees with the record, and for a few it counts the steps with no misjudgment at the rule level. "Levels replayed" counts the levels whose recorded winning run Lean replays from the first screen up to the winning frame, whose step is checked by the model's goal test (formal replay coverage).

Game	Lv.	Cert. steps	Obj. types	Rules (Lean)	Rules total	Steps expl.	Levels replayed	Lean thms	Concepts (withdrawn)
ar25	8	253	6	5	8	253/253	8/8	43	17 (3)
bp35	9	374	3	10	11	348/374	9/9	41	26 (4)
cd82	6	70	7	10	11	657/661	6/6	81	44 (13)
cn04	6	261	4	8	9	742/745	6/6	51	22 (1)
dc22	6	1,037	8	14	17	2664/2664	6/6	77	48 (6)
ft09	6	80	6	4	7	168/168	6/6	50	15 (6)
g50t	7	288	10	11	13	280/280	7/7	64	20 (1)
ka59	7	421	9	10	11	3959/3959	7/7	67	22 (8)
lf52	10	905	12	10	11	896/905	10/10	64	47 (6)
lp85	8	79	6	4	6	199/199	8/8	47	23 (4)
ls20	7	368	12	18	21	1460/1465	7/7	42	22 (3)
m0r0	6	215	8	8	9	485/486	6/6	67	26 (5)
r11l	6	81	6	8	9	183/184	6/6	65	26 (4)
re86	8	547	7	8	12	2153/2153	8/8	42	40 (4)
s5i5	8	461	5	17	19	2617/2618	8/8	70	49 (9)
sb26	8	124	6	12	16	424/424	8/8	53	20 (1)
sc25	6	146	5	11	18	382/401	6/6	31	27 (1)
sk48	8	263	6	4	7	1364/1407	8/8	66	39 (8)
sp80	6	138	3	6	10	353/371	6/6	25	58 (23)
su15	9	90	5	12	16	734/904	9/9	62	37 (15)
tn36	7	125	5	4	7	292/292	7/7	28	33 (9)
tr87	6	119	6	4	7	429/429	6/6	50	13 (3)
tu93	9	187	8	8	10	560/566	9/9	76	17 (0)
vc33	7	167	9	7	10	535/538	7/7	75	14 (0)
wa30	9	649	8	7	8	1515/1515	9/9	35	37 (6)
Total	183	7448	170	220	283	23652/23961	183/183	1372	742 (143)

(every real transition is possible in the model, and every real goal maps to a model goal), then (Aineto & Scala, 2024; Clarke et al., 1994)

$$\text{Unsolvable}_{\widehat{\mathcal{M}}}\big(h(s_0)\big) \implies \forall\, \bar{a}.\, T^{\bar{a}}(s_0) \notin G.$$

A model learned from observations satisfies the former property but not the premise of the latter. AWM therefore states its impossibility results as Unsolvable of the inferred model. The failure in §4 is a violation of the over-approximation premise in the unsafe direction: an abstraction that forbids a route the engine allows. §4.3 reports where this premise fails and how the agents repaired the rule at fault.

A.6 Abstraction across games. Table 2 lists, for each game, the size of the concept book, of the derived layer and of the final rule layer, the shared rule that absorbed the most concepts, and the comparison on corrupted steps where one was made. In 24 of the 25 games the per-game report states at least one many-to-one merge; `cd82` gives only the reason for a rebuild. In the per-game reports, which describe more merges than the largest one listed in the table, translation rules recur in 16 games, win conditions written as one equivalence in 10, counters and budgets in 7, switches in 6, and permutations in 3 (`lp85`, `dc22`, `ls20`). Where the book and the final rules were compared on the same corrupted steps (14 games), the rules reject more in 12, of which 10 reject a superset; one comparison is saturated (`tr87`); and in `dc22` the rules reject 2 fewer of 400, 26 steps being rejected only by the book. Condition (**??**) flagged 186 rules across the 25 games; none was removed on that ground alone, and each stays flagged in its game's record. Most reports give the reason for keeping,

usually that the corrupted set is too coarse to single the rule out, and those of `lp85`, `dc22`, `g50t` and `re86` show each flagged rule rejected on its own by a source-code mutant.

Table 2: Abstraction in each game, from the per-game reports (bench `f607dfc0`). *Book*: concepts at the start of abstraction; *Derived*: entries in the model's derived layer, which computes composite quantities from the object layer (`derive/`); *Rules*: rules in the final layer (these per-game counts sum to 287, against the 283 of §3, which are read at the paper's build). *Merged*: concepts the largest shared abstraction absorbed. *Rejected*: corrupted steps rejected by the book and by the final rules, with the size of the set in brackets; the set differs across games, and in `cn04`, `dc22`, `sp80` and `tn36` it is the full corrupted corpus (400 to 1,804 steps), not the at most 120 used for elimination, and in nine games only the rules side counts steps the object parser refuses. *Change*: book against final rules; "int." marks games that only compare intermediate rule versions. [a]Counts intermediate rules. [b]One concept per direction split into three rules. [c]Absorbed into a group of seven rules. [d]Not a superset: the book alone rejects one reset step. [e]The report does not state it; concepts minus withdrawn concepts in Table 1, which gives the Book value of every other game.

Game	Book	Rules	Derived	Largest merge	Kind	Merged	Rejected	Change
`ar25`	14	8	6	`push_moves_selected`	translation	4	—	int., equal
`bp35`	22[e]	11	2	`sideways`	translation	3	—	int., equal
`cd82`	31	11	6	(rebuilt)	—	—	115/117 (120)	up
`cn04`	21	9	5	`move`	translation	9	377/399 (400)	up
`dc22`	42	17	7	`move`	translation	6	329/327 (400)	down
`ft09`	9	7	4	`level_up_iff_all_done`	win condition	2	116/118 (120)	up
`g50t`	19	12	4	`walker_patrols`	patrol	4	—/95 (120)	—
`ka59`	14	11	7	`push_shoves`	translation	4	92/119 (120)	up
`lf52`	41	11	3	`keys_push_carts`	translation	9	78/117 (120)	up
`lp85`	19	6	5	`press_permutes`	permutation	11	117/119 (120)	up
`ls20`	19	21	6	`walk/slide/blocked`	translation	4[b]	105/118 (120)	up
`m0r0`	21	11	5	`pieces_move`	translation	6	—	int., up
`r11l`	22[e]	9	8	`absorb`	region fill	3[a]	—	int., equal
`re86`	36	12	3	directions a1–a4	translation	4	—/119 (120)	—
`s5i5`	40	19	4	void-step group	other	9[c]	—/117 (120)	—
`sb26`	19	16	3	`win/submit`	win condition	2	116/119 (120)	up
`sc25`	26	18	10	`resize`	other	4	118/119 (120)	up[d]
`sk48`	31	7	2	`rope_slide`	translation	2[a]	—	int., equal
`sp80`	35	13	7	`arrow_shifts_selected`	translation	6	1580/1778 (1804)	up
`su15`	22	16	5	`click_lands_fresh`	other	2[a]	—	int., equal
`tn36`	24	7	4	`piece_follows_program`	program	7	1390/1424 (1434)	up
`tr87`	10	7	3	`frame_rest`	static	3	120/120 (120)	equal
`tu93`	17	10	4	`player_moves`	translation	2[a]	—	int., equal
`vc33`	14	8	5	`press_button`	other	3[a]	—	int., equal
`wa30`	31	10	9	`moles_act`	pathfinding	10	85/120 (120)	up

A.7 Unexplained steps. Of the 23,961 recorded steps, 309 are not explained by the rules (§3). Table 3 sorts them by cause. Only 52 are wrong predictions, 51 of them in `su15`; on the other 257 the model gives no verdict. The largest groups are undo, which needs history the screen does not hold (100), and steps that reset or change the level (78). State the screen does not show accounts for 70: a facing direction in `sk48`, a piece that drifts out of the visible window in `bp35`, overlapping pieces in `sp80`, fog in `ls20`. Several pieces acting on each other in one step account for 26, all in `su15`.

Table 3: The 309 recorded steps the rules do not explain, by cause (bench `f607dfc0`). Counting follows each game's report: Lean step agreement in most games, the rule layer in `s5i5` and `m0r0`.

Cause	Steps	Games
Undo: needs history	100	`su15` 92, `bp35` 8
Reset, new game or level change	78	ten games
State not visible on the screen	70	`sk48` 35, `bp35` 18, `sp80` 12, `ls20` 5
Several pieces interact in one step	26	`su15`
Trigger not identified, branch left out	21	`su15` 17, `sp80` 4
Attributed but unfixed error	9	`su15` 8, `s5i5` 1
One side is a win screen the parser cannot read	5	`sc25` 3, `m0r0` 1, `sk48` 1

B The budget-sealed instance: construction, protocol and known gaps

This appendix records what §4 leaves out: the construction order, the protocol details, every reading of ours that was overturned while the instance was built, and the batches behind Table 3 and Figure 4. They are kept because an impossibility claim is only as good as the audit behind it. Exploratory batches that the main text does not use are omitted: a preliminary attempt, a replication batch, a wording batch, a tools-only control, a second Codex model and an upper-approximation protocol, all run before the method asked for closed proofs, and a completion round and a control on a corrected engine; their records are kept with the run archive.

B.1 Order of construction. The order was fixed in advance and not reversed: locate the field that stores the budget and confirm that editing only it changes only the budget; build the abstract models and cross-check them against the engine edge by edge; search for $B^\star$, the cost of the cheapest win; derive the sealed level at $B^\star - 1$; certify $B^\star$; self-audit; and only then stand up the runs. §4 relies only on the engine-level exhaustive search for $B^\star$; our own reference certificates and their Lean statement are not used as evidence there, because they too would need a level-5-specific bridge from the engine to the abstract model (§4.3 and §B.11). The cross-check precedes the search because using the engine as an oracle costs milliseconds per edge while the abstract model costs microseconds, and the later stages need the model anyway. The proof precedes the runs because a machine search reporting "exhausted, no solution" only says so *under the semantics we wrote*: one wrong clause makes the instance fake, a fake instance does not announce itself, and every run's reading would be void.

B.2 Configurations.

- **bare**: task and interface with the coding agent's ordinary tools; no required AWM procedure or supplied formal skeleton. The original runs used Claude and Codex (`gpt-6.1-sol`); the fresh-arena batch used Claude and `gpt-6-astra`, three runs each (§B.17). This is the control: whether it decides unsolvability, and what it offers as warrant, is what the experiment measures.
- **framework**: the AWM procedure, a generic Lean skeleton, and tools for recording predictions and checking rules, but no ready-made model. One original run, three fresh-arena runs, and the closed-proof batches of §B.18.
- **model**: the framework plus an existing executable concept book, to be tested, repaired and extended. One original run and three fresh-arena runs.
- **bare-asked**: the bare workspace, but the interviews also request formal evidence and offer model-relative verdicts. One original run.
- **solvable side**: the same board with level 5's cap restored, as the non-vacuity gate for the configurations themselves.

Arena directories carry opaque names with no marker of configuration or side, the arenas live outside the project tree, no ground-truth file enters an arena, and cross-run memory is cleared before an independent run starts; state and files persist between attempts of that run, with the entering file count required to be 0. Runs interrupted by machine failures (two restarts during the batch) were rerun from the start; their partial arenas were archived and are not counted. All runs reported in §4 use the public rule code; the corrected engine is used only in a level-4 control of three bare `gpt-6-astra` runs, which is not reported here.

B.3 Two harness gaps, closed before the runs. The blinding lists inherited from the level-4 instance did not cover this level. The leak-word list carried that level's cap but not this level's constants, and the forbidden-file list carried the files of the level-4 sealed level but neither this level's files nor the artifacts built for it (the Lean development, the closure certificates, the construction record and the certifier). Either gap could have let a ground-truth file sit in an arena unnoticed. Before the runs, the lists were extended (sides $2 \to 3$, leak words $21 \to 34$, forbidden files $15 \to 34$) by runtime injection, which left the earlier harness byte-identical because its readings were still in use elsewhere. Afterwards both arenas were scanned and contained no ground-truth file. The arena name was also made opaque, because the earlier scheme wrote the configuration and the variant of the sealed level into a path the agent could read.

B.4 Readings that were overturned. (i) The teleport landing coordinates were first read one cell off, because the sprite table was read with an offset that the engine does not apply at landing. Re-measuring three levels on the engine gave zero offset in all three, and the earlier coordinates were voided. (ii) The first cross-check compared only position, size, facing and count. It agreed on 1,182 steps with no mismatch and still missed the two-specimen mechanic. It caught the defect only after a predicate for "this color is still on the board" was added, which is why later cross-checks compare the erase mask as a field. (iii) The fuel refund is 10, not 9. The net change of -9 that a player reads off the budget bar is the refund followed by the action's own increment. The framework run changed a correct rule into an incorrect one by reading the bar. A guess that this error explained the bound being one too high was refuted by the machine: recomputing $B^\star$ with the refund set to 9 gives 36 in both models. (iv) After a single smoke attempt we wrote that the risk of never reaching level 5 had been retired. The first full attempt then stopped at level 4, with more actions than the smoke attempt had used, so the claim was withdrawn and replaced by the weaker, true one: the level-4 gate is itself a filter and consumes attempts. (v) Of six predictions registered before the runs, three were wrong. One was the prediction that the framework run's lower bound would be too large; its modeled bound equals the engine's $B^\star = 36$. That agreement is numerical, not a transferred theorem, because the run's model differs from the engine in the teleport landing, in where the small body may grow back, and in charging the winning move; under the engine's accounting the run's own model admits a win at cap 35 (§B.6). (vi) Cost was estimated twice and missed twice, once low and once high by a factor of 2.5–3.7. The estimate extrapolated linearly from the first attempt, while the runs stopped exploring: actions per attempt fell from 242 to 0, and two of the eight attempts account for more than half the spend.

B.5 Engine ground truth. Table 4 summarizes the two sealed levels.

Table 4: The two sealed levels. Costs are in counter units; "gross" adds back the fuel refund of 10. Reachable states are those of the engine at the sealed cap, none of which is won.

Level	Public cap	Sealed cap	$B^\star$ (gross)	Reachable states	Win at $B^\star$
4	35	18	19 (29)	656,715	27 actions
5	65	35	36 (46)	1,346,362–1,354,849[a]	42 actions

[a] Per opening; the opening facing carries over from the previous level and cannot be read from the screen, so level 5 was searched from all four (Appendix B.8).

The search runs on the sealed engine itself. Its actions are the four moves and the nine panel coordinates that the engine exposes, clicked one at a time; recipes are not abstracted. The state key is every sprite except the counter bar, the panel display and the spell icons (name, position, scale, rotation, pixels, visibility), together with the panel matrix, the facing, the animation flags, the teleport target index and the game state. The counter is not in the key: each key keeps the least count at which it is reached, in a label-correcting search where a refund may reopen a state. This is sound because the counter enters the engine only through its unit increment, the refund $\max(0, c - 10)$, the loss test *count* $>$ *cap*, and the display. A fast state copy agreed with a deep copy on 6,000 steps, a rerun after an externally killed first run reproduced the state and edge numbers of every count from 0 to 23, and clicks on all 4,096 screen coordinates in 22 sampled positions had no further effect.

The opening facing is a game-level attribute: a level reset does not reset it, the opening frame does not show it, and all four facings occur in the service. The search was therefore run from each opening:

opening	states, cap 35	engine edges, cap 35	wins	states, cap 36	wins
the one `set_level` produces	1,346,362	17,515,680	0	1,371,613	1
each of the other three	1,354,849	17,626,011	0	1,380,256	1

The 42-action win is

```
D D R c1 c4 U c7 R R R R D R R R U U U L c1 c3 c5 c7
  U U U U R c1 c3 c5 c7 c1 c4 c7 c0 c1 c4 U U U U;
```

the searches from the other openings return 42-action wins that differ from it only in where one upward move falls among the first clicks. It is 25 moves and 17 panel clicks that complete five recipes.

Each completing click is charged one extra unit, and the final move into the goal is not charged at all and is allowed when the count equals the cap, so the gross charge is 46 and, after the refund of 10, the count peaks at 36. The original engine with its cap set to 36 accepts the same sequence.

B.6 The original framework and model runs against the engine. The original framework run's `every_win_costs_36`, `no_win_within_budget` and `plan36_wins` have no hypotheses, use no axioms beyond the standard three, and are closed by a certificate of 3,432 entries. Their definitions differ from the engine in five places:

rule	model	engine	direction
teleport landing	one cell for both body sizes	large and small bodies land in different cells	stricter for the small body
growing back	small body grows back only inside one ring; elsewhere a no-op that costs 5	grows back wherever a 4×4 fits within a fixed offset	stricter; the engine's only optimal win uses it
winning move	costs 1	costs nothing, allowed at count $=$ cap	stricter by 1
budget reading	may act only at count ≤ 34	loses when count $>$ cap	off by one
move step	small body may move 1 pixel	small body moves 2 or 0; large body 4 (2 when blocked)	literal mismatch

Varying the bound shows how far the theorem reaches. Raising `every_win_costs_36` to 37, or strengthening `no_win_within_budget` from 35 to 36, which is what impossibility at cap 35 needs under the engine's accounting, does not compile; adding that the first 32 steps of `plan36` cost 35 and the 33rd step is a winning move does compile. Under the engine's accounting, then, the run's own model admits a win at cap 35. Over all reachable states at cap 35 and all 13 actions, 17,194,487 non-losing engine edges, grouped into 32,523 abstract queries and run through the agent's own `step`, the model cannot reproduce 59,340 edges, and 4,697,218 edges violate the certificate's potential inequality, in the teleport and grow-back classes because the engine is cheaper than the certificate allows. The shortest counterexamples, replayed in a fresh engine process, are a small-body teleport and a small-body grow-back. Translated into engine actions, `plan36` dies on step 42 at cap 35 and fails to clear the level at cap 36 and at cap 60, stopping as a small body, because the engine's small-body teleport lands where the model does not place it. The original model run's `L5Budget.lean` states its conclusion under four geometric and two budget premises, none proved in Lean. The move-count premise fails on the engine's own win, which takes the fuel with fewer moves than the premise requires, and under the run's own pricing both budget premises are false; raising the budget premise to the engine's value makes the file fail to compile.

B.7 Companion seal at level 4. The same game sealed at level 4, cap 18, has $B^\star = 19$: at cap 18 the engine has 656,715 reachable states and 8,769,293 edges and no win; at cap 19 a 27-action win exists. The original runs there ended nine `undecided` and three `impossible` out of twelve. In a re-run of seventeen runs, three of five bare Claude runs ended `impossible`: two of three under the original wording, none of one under a neutral wording, and the one run whose task said that some levels may be unbeatable. All three are right about cap 18 for wrong reasons. Their load-bearing bounds, at least 28 actions, at least about 30, and about 110 bar rows for a small-body route, are refuted by the engine's 27-action win, which grows back onto the fuel tile and collects it and then fires with the large body, the two moves the runs had ruled out. The same bounds would declare the solvable boards at caps 19–21 impossible. Three tooled runs answered `impossible_in_model_proved` with Lean that compiles with clean axioms; none transfers. One rests on a premise that is false on the engine, and the other two reach a threshold equal to $B^\star$ only because two errors in opposite directions cancel. Over the original and the re-run, five of twenty bare runs outside the hint variant end with an impossibility verdict (six, counting a `gpt-6-astra` run that asserted it at attempts 5–7 and ended `undecided`), and none leaves a checkable artifact. The level-4 proofs that §4.2 relies on come from the three framework runs of §B.13, not from these three tooled runs.

B.8 Instances, agents and the Codex harness. *Level 5 ground truth.* The ground truth is an exhaustive search on the real engine, not a model of ours. The level's opening facing is not fixed: it is carried over from the previous level, is not reset with the level and cannot be read from the opening frame, and all four facings occur in the service (Appendix B.5). We therefore searched from all four openings. At cap 35 every reachable state is explored and none wins: 1,346,362 states linked by 17,515,680 engine edges from one opening, and 1,354,849 states linked by 17,626,011 edges from each of the other three. At cap 36 every opening admits a win. A 42-action winning sequence, replayed in a fresh engine process, clears the level at cap 36 on its 42nd action and ends the game at cap 35 on its 41st. The optimum is therefore $B^\star = 36$ from every opening, and the sealed cap is one unit below it. It does not depend on a refund rule of the public engine that matters on level 4 (§4.2): our model of the level, cross-checked against the engine, gives 36 under either form of that rule.

Run counts. We name the level-5 batches as follows. The *original* batch has one eight-attempt run per configuration and, for each bare agent, one further twenty-attempt run (Table 5); the *fresh-arena* batch is that of Table 3 (§B.17); and the *closed-proof* batches are those of §B.18. The runs are read as a record of what happened, not as rates: whether a bare agent asserts impossibility within eight attempts varies between batches (§B.9).

Agents. All agents observe screens and submit actions through the same interface in isolated workspaces, and each attempt ends with an interview that records a verdict and its argument. The **framework** agent receives the AWM procedure, a generic Lean skeleton and tools for recording predictions and checking rules, but no game rules. The **model** agent also starts from an existing concept book, which it is told to test, repair and extend. **Bare** agents receive only the task and the interface, with the ordinary tools of a coding agent. A **bare-asked** control keeps the bare workspace but also requests formal evidence. The original runs used one eight-attempt run per configuration; Claude runs share the base model family of the AWM agents, and the Codex runs use `gpt-6.1-sol`. We later found in the agents' own session logs that Codex runs made with our harness before a fix ran in a read-only workspace from their second attempt on, so the agent could not write files; both `gpt-6.1-sol` runs on this level are among them, and we mark them where they are used. Later Codex runs use `gpt-6-astra`, because `gpt-6.1-sol` was not available to us.

Codex runs, checked. The Codex runs used in §4 were checked turn by turn in their session logs. On level 4 the three runs on the public engine ran `gpt-6-astra` with a writable workspace in every turn; on level 5 the twenty-attempt rerun did at every attempt. The two earlier `gpt-6.1-sol` runs on level 5 had a writable workspace only in their first turn (1 of 17 and 1 of 42), and so did five `gpt-6.1-sol` runs of an earlier level-4 batch, which we therefore do not use.

B.9 Level 5: what play alone gives, in detail. Under the original wording neither eight-attempt bare run of the original batch, Claude or Codex, asserts that the level is impossible; in the fresh-arena batch all three bare Claude runs do (§B.17). Longer play changes the first reading: the twenty-attempt Claude run of the original batch switches to `impossible` at attempt 11 on a count of 47 units against 45, after writing its own simulator of the level and searching it; its 47 exceeds the true gross cost of 46 by exactly the winning move, which it charges and the engine does not. The twenty-attempt Codex run of the original batch stays undecided, but it could not write files after its first attempt, so it cannot show what longer play does; a rerun on `gpt-6-astra` with a writable workspace stays undecided through all twenty attempts and the final interview (confidence 95). On the companion seal at level 4, where the agent meets fuel for the first time, two of four bare Claude runs under the original or a neutral wording, and the one run given a hint, assert impossibility, and all three rest on lower bounds that the engine's 27-action win one unit above the cap refutes (§B.7). Both original bare agents clear the level when its budget is restored to 65, so the gap is not one of skill at the game.

B.10 Level 5: the AWM runs, in detail. *What AWM proves.* In the original framework run the agent first reports impossibility under its model at attempt 3, and the model agent at attempt 1. The framework agent builds a transition model and a potential-function certificate of 3,432 states, closed by the kernel, and proves in Lean

> `every_win_costs_36`: every winning action list has modeled total cost at least 36;
> `plan36_wins`: a modeled winning action list has total cost exactly 36.

Table 5: Original runs on the sealed level, one row per agent. *Verdict* is the final verdict; *First* is the attempt of the first impossibility verdict (unqualified for bare agents, under the model for AWM and bare-asked). *Form* is the kind of argument behind it. *Non-vacuous*: the formal statement is not trivially true (a win exists in the model). *Value*: the stated bound equals the engine's $B^\star = 36$. *Transfers*: the conclusion holds on the engine. Tokens are recorded input plus output up to the first verdict, in millions.

Agent	Att.	Verdict	First	Form	Non-vacuous	Value	Transfers	Tokens
Bare, Claude	8	undecided	—	none	—	—	—	—
Bare, Claude, continued	13	undecided	—	none	—	—	—	—
Bare, Claude, second run	20	impossible	11	own simulator, unchecked search	—	no (47)	no	11.9
Bare, Codex, second run[b]	20	undecided	—	none	—	—	—	—
Bare, Codex, rerun[c]	20	undecided	—	none	—	—	—	—
Bare-asked, Claude	8	in model	3	Lean, arithmetic	no	—	no	8.0
AWM model, Claude	8	in model	1	Lean, arithmetic on premises outside Lean	—	—	no	11.3
AWM framework, Claude	8	in model	3	Lean, potential-function certificate	yes	yes	no[a]	31.2

[a] The value agrees only because two departures from the engine cancel; after the transitions the bound depends on were tested against the engine and the model repaired post hoc (§B.11), the same bound passes an exhaustive engine check. The model agent's premises are false on the engine (Appendix B.6). [b] `gpt-6.1-sol`; read-only workspace from attempt 2, so it could not write files. [c] `gpt-6-astra` with a writable workspace, checked at every attempt in its session log.

The first is the potential-function argument of §2.4, of the kind Figure 5 draws for a later run: a quantity that no action can lower by more than its cost, so that the cost of any win is bounded below. The second shows the bound is tight in the model. Both are theorems about the agent's inferred model, and they reach the game only as far as that model contains the engine. It does not. Of its five departures from the engine (§B.6), three bear on its routes: it sends a small body's teleport to the wrong cell, it lets a small body grow back only inside one ring, and it charges one unit for the winning move, which the engine does not count. The engine's only optimal win grows back outside that ring, a move the model forbids, and replayed on the engine `plan36` does not clear the level at cap 36 or even at cap 60. The agreement of the bound with $B^\star = 36$ is numerical, not a transferred theorem (Appendix B.6). The model agent's arithmetic proof in `L5Budget.lean` likewise rests on premises outside Lean, which are false on the engine.

B.11 Checking what the bound depends on. A Lean certificate establishes that a conclusion follows from the encoded transition system. It does not establish that the encoded system describes the environment. The framework agent's bound shows the difference. We translated its Lean model into Python, checked the translation against Lean's own evaluation on all 32,523 abstract transitions found by the exhaustive engine search of §4.1 with no disagreement, and compared the model with the engine. Two of its departures act on the bound. The wrong teleport gives the model a path the engine does not have, and the charge on the winning move adds one unit to every win. They offset. The modeled bound equals the engine's threshold, but by the engine's accounting the unrepaired model does not give 36: charging the winning move as the engine does lowers its bound to 35, along a path through the refuted teleport.

Testing the region below the bound. A model lower bound motivates checking its below-bound transition region. Agreement on that model-derived region alone does not ensure that every cheaper engine execution is represented: the engine may reach states or allow actions the model omits. Transfer also requires complete coverage under the abstraction, action-set, and initialization assumptions of the engine search. For the framework agent's model this region holds 20,853 (state, action) pairs. For each pair we started a fresh game, reached the state by real actions, and executed the action on the engine; 421 pairs could not be reached, and the region also contains 1,144 transitions that die at the cap. Completeness therefore rests on a final exhaustive check, not on the region. The procedure used about 470,000 engine actions and recorded 3,184 disagreements (teleport 1,607, resizing 1,457, turning in place 48, multi-cell moves 26, other 46).

The preregistered repair only adds observed transitions. Under it the bound stays 36, but that 36 still includes the charge on the winning move. In an analysis added after the preregistered one, we replaced each refuted transition with the observed one and charged the winning move as the engine does, taking the winning move from an observed run at cap 36. This gives a bound of 36 again, and the repaired model passes an exhaustive check against all 32,523 abstract engine transitions within cap 35 with no violation; every transition on its cheapest modeled win also occurs in the engine. A second framework model, from a batch not reported here, gives a similar picture in its own cost units. The bound its author stated (a gross cost of 46) is already false in its own model, whose search gives 43 without refuelling and 47 with it. Its net bound under the preregistered repair is 37, which again includes one unit for the winning move; 790 repairs leave it unchanged, and 1,001 region pairs could not be reached. With the same post-hoc repair it gives 36 and passes the same exhaustive check. Two limits apply. Over the transitions within the cap, this certificate states the same fact as the exhaustive search; its value is that it is derived from the agent's own model and can be checked independently. And the repair writes observations back into the model; it does not learn general rules.

B.12 The blind grading, and Lean against an equally checked program. *The blind grading.* Put side by side, the agents differ less in what they conclude than in what they leave for a reader to check. Graded blind against a criterion that requires every load-bearing premise to be checkable on the engine, none of the 65 verdict arguments, on either side, transfers to the engine; the most common error, which occurs on both sides, is charging the winning move. The difference between the agents is therefore not that one reasons correctly about the game, but that one leaves objects a reader can check. Bare agents leave prose and hand counts. The framework agent leaves an executable model that can be compared with the engine edge by edge, Lean that compiles with clean axioms, and a witness that can be replayed on the engine. Its theorems hold in its inferred model; whether such a model contains the engine is itself checkable, and in every case we checked it did not.

Lean against an equally checked program. On the framework agent's model and a second one, we injected errors into the certificate generator, the model definition and the theorem statement (Appendix C). Lean's kernel rejected every invalid certificate, and so did a Python checker of the same conditions; a Python search that only prints its result accepted five wrong bounds. Both routes accepted the same wrong models, and both accepted vacuous bounds until a reachability witness was required. In the agents' own developments, the key conclusions rest on premises outside the kernel, one of them false. Lean's distinct contribution here is auditability: it shrinks what must be trusted to the model and the statement, and writes the premises into the theorem. Reliability about the environment then comes from checking that model against it, and the theorem says exactly what to check. Appendix C lists the deviations from the frozen protocol.

B.13 Level 4: the further batch. Besides the runs of Appendix B.7, we ran a further batch under the original wording: three eight-attempt runs for each of three agents (Table 6). No bare Claude run asserted impossibility, in any attempt or in the final interview. A longer, thirteen-attempt Claude run asserted it at attempt 9, returned to `winnable` at attempt 11 and asserted it again at attempts 12 and 13, on a gross cost of 32 against the true 29. Two of three bare `gpt-6-astra` runs, whose session logs show that model and a writable workspace throughout, asserted it at attempt 8, each on a cheapest cost of 32 (one in half units, 64 against 56). One returned to `undecided` in the final interview; the other kept the verdict and added that a search under a deliberately loosened rule set "still requires 29", the true optimum. The three framework runs first reported impossibility at attempts 1, 2 and 3, one of them returning to `winnable` at attempts 4 and 5, and each submitted Lean that compiles with clean axioms. Compiling is not bounding the game. Rebuilt with other caps, the original framework run's development (first impossibility verdict at attempt 3) keeps its impossibility up to cap 26, so its model is stricter than the engine by eight units, while the original model agent's development (first verdict at attempt 2) holds up to cap 18 and fails at 19, the engine's threshold. As on level 5, a reader can compute where each Lean result stops being true; a bare run leaves nothing that can be tested this way.

B.14 Hint round. Table 7 lists, for the five AWM runs on level 4 whose bound we can read, the refill rule each found in play and after hints, and the least winning cap its model gives.

Two level-4 framework runs had refill rules stricter than the engine's: the one that proved 20, and the earlier framework run whose development holds up to cap 26 (least winning cap 27). We resumed

Table 6: The level-4 seal: our further batch, eight attempts per run, original wording. *Asserted*: runs with an unqualified `impossible` (bare) or an impossibility verdict (framework) in some attempt, and the attempts at which they first did. *Stated*: the cost and budget a bare run gave when it asserted; the truth is 29 against 28. Appendix B.7 reports other runs on this level.

Engine	Agent	Runs	Asserted	Stated
public	Bare, Claude	3	0	—
	Bare, `gpt-6-astra`	3	2 (8, 8)[b]	32 against 28, both; one adds 29 under a loosened rule set
	AWM framework	3	3 (1, 2, 3)[a]	Lean, clean axioms

[a] One run returned to `winnable` at attempts 4–5. Rebuilt with other caps, the proof of the run whose Lean bound is 20 holds up to cap 19 and fails at 20; the Lean file of the run whose bound is 19 proves that no list wins up to cap 18 and gives a win at 19, so its range is 18 (its theorems cover every cap up to 18, and the file recompiles); the third run's universal theorem rests on a check that did not finish compiling, so its range was not measured. For the original framework run on this level the impossibility holds up to cap 26. [b] One of these runs returned to `undecided` in the final interview.

Table 7: Level 4: the refill rule and the least winning cap in the five AWM runs whose bound we can read. The engine refunds when a small block covers the fuel, when a large block's top-left quarter covers it after a move, and when any part of a large block covers it after a grow; its least winning cap is 19. Hints: R1 "your model is stricter than the engine", R2 names the refill rule, R3 says the outcome depends on how the block arrives and on its size. *Least cap* gives the value before the first hint listed and after each hint in turn: for the run with R1, R2 it is 20 before hints, 20 after R1 and 19 after R2; for the earlier run it is 27 before hints, 26 after R1 (given in an earlier session), 26 after R2 and 23 after R3.

Run	Refill rule from play (attempt)	Hints	Rule after hints	Least cap
Model, earlier	grow collects (4); walking never does (6)	—	—	19
Framework	small block collects (2); amount 10 (6); grow collects (7)	—	—	19[a]
Framework	small block collects (2); any overlap collects, noted as looser than the game (7)	—	—	19
Framework	small block only	R1, R2	large block: top-left quarter, after a move or a grow	20, 20, 19
Framework, earlier	small block only	R1, R2, R3	the engine's rule	27, 26, 26, 23[b]

[a] From an exhaustive search of the model in play. [b] The third hint round was cut short when the machine ran out of memory; its final reply was recovered from the session record and its proof file compiled by us.

each session with hints in three grades, fixed in advance and none giving a number or stating the rule. R1: the engine has a cheaper winning route than the model allows, so at least one rule forbids, or charges more for, a move the engine allows. R2: the discrepancy is in the rule for collecting the fuel refill, and there is a situation in which the engine gives the refill and the model does not. R3: whether the refill is collected depends on how the block arrives on the fuel, by walking or by changing size, and on its size. Each hint was followed by the same request: the run could play the level again on a restarted service, and then had to say which rules it changed and on what evidence, update its proof file so that one kernel-checked theorem over every action list of its model, with only the standard axioms and every unverified rule an explicit hypothesis, states that no list wins within the cap it measured, and report the least cap at which its corrected model, with every rule as its evidence supports it, admits a win. A run stopped at the first grade at which that cap was 19, and after R3 in any case. For the earlier run, R1 had been given before, in slightly different words ("forbids at least one move the engine allows"), and we started at R2. We judged the refill rule *fully found* if the corrected model refunds a grow that covers the fuel anywhere and a walking large block only when its top-left quarter covers it, and *partly found* if it has only one of these.

The run that proved 20 played again after R1 and changed four rules, none of them the refill: blocked moves turn the block, the pattern must match exactly, the switch needs a vertical facing, and a win

counts before the cap is checked. The last matches the engine, which neither charges nor checks the move that reaches the goal, but it did not lower the bound: its least cap stayed at 20. After R2 it ran six targeted experiments on the fuel, among them a large block walking onto it from two directions, a shrink and a grow on it, and concluded that a large block collects the fuel when its top-left quarter covers it, after a move or after a grow. It also tested its R1 change by playing its cheapest route at cap 18, saw the game end one move short of the exit, and reversed that change; the move that ended the game was not a winning one, because its model counts a move onto the exit as the win, while the engine counts the move that bumps into it. Its least cap is 19, with a winning list at 19 and a proof of no win below; we compiled the file (clean). The walking half of its rule matches the engine; for grows it uses the same top-left test rather than the whole body, which is stricter than the engine at one position and does not change the bound. We count it as partly found.

The earlier run, after R2, found that a grow whose large block covers the fuel even in part collects it, that the refill is worth 10, and that where a grown block lands depends on its facing; its least cap stayed at 26, because a walking large block still never collected. After R3 it ran further walking experiments with the large block, found two that collected, and wrote: a walk of either size collects iff the block's top-left unit cell is on the fuel, a grow collects iff the new large block covers it anywhere, and the refill is 10. This is the engine's rule, fully found. Its least cap fell to 23, with `no_win_upto_22` and a win at 23. It is not 19 because its rule for where the switch can fire still allows only a small block at fixed positions, while the engine also fires with the large block. This round was cut short. Machine-wide memory ran out while a Lean process of another experiment held 13.9 GB; our driver and the game service stopped at about 00:53, while the run continued to its last Lean edit at 01:04 and ended without the closing lines. We recovered its last reply and the rule from the session record and the Lean files, and compiled its proof file ourselves (144 s, clean). Since then the service runs as a separate process.

In both runs a hint that named the rule, without its content, was enough for the run to design experiments that separated the cases within one round; the weaker hint was not. Two caveats apply: we wrote the hints after reading the engine's code, though the rule each names is also the one at which replaying the engine's winning route through the run's model first fails (§B.15); and there are only two runs.

B.15 Material condensed from Section 4. Several analyses that Section 4 cites are given in this appendix: the level-4 hint round in §B.14, the closed-proof batches and their audit against the engine in §B.18, and the level-5 repair round in §B.19.

From rule objects to a proof (full text). An AWM agent's model is a set of Lean objects (§2.4). On level 4, one framework run has, by its seventh attempt, a state record with five fields (whether the block is large, whether the barrier is open, whether the fuel is still on the floor, and the block's row and column) and an action type with the four moves and the spells as single macro actions. The transition function dispatches to one definition per rule, and each definition records the attempt and the transition it was read from. Rules the agent could not pin down are not guessed. They enter the model as hypotheses or as sets of admissible outcomes: a hypothesis `hopen` that a shot opens the barrier only from the column where it was seen to, and a list of possible landings for a resize, any of which the rule may choose.

Checking the proofs. Because a bound is stated over named transitions, it can be checked against the engine step by step. The engine's winning list at cap 19 cannot be a run of cost 19 in the model of the run that proves 20, so replaying it through that model stops at, or overcharges, some step, and that step names the rule at fault. We ran this replay afterwards for the two runs of the hint round, each with the model it had when the hint naming the refill rule was given: in both, the first step the model does not reproduce is the grow onto the fuel, which it does not refund; in the earlier run the next is the shot fired by the large block, which its model lets only a small block fire. In an earlier pair of runs on this level, recompiling each development at other caps showed where its claim stops: the model agent's impossibility holds exactly up to cap 18, the engine's threshold, and the framework agent's up to cap 26, eight units too far (Appendix B.13). None of the bare arguments we audited comes with a model and a certificate that can be checked in this way.

Future work: new goals. With a reusable transition model the goal can change without relearning the dynamics. We plan to test whether this saves interaction on new reachability tasks, with tools and budgets matched; the present experiments say nothing about it.

B.16 Interview prompts. After each attempt the harness sends the interview below, in the same session as the play. All runs receive questions 1–3 and the verdict lines; AWM runs also receive the formalization questions, in the version of the harness they ran under, and bare runs receive a verdict line without the model-relative answers. The level is then reset with the retry message. After the last attempt the final interview replaces questions 1–3 by its own four questions and keeps the same verdict lines. The texts are reproduced verbatim.

Questions 1–3, every run.

```
That attempt failed. The game will reset this level, and you may get another attempt.

Answer the following. Keep it to what it needs; do not pad.

1. What happened on the level you were on when the attempt ended, and what do you think made
   it fail? Explain in plain prose, as if to someone who did not watch you play.

2. Do you still see a way to beat this level?
   - If you do: say what it is, and what you would still need to check to be sure of it.
   - If you no longer do: say why not. State precisely what you are claiming, give the
     argument for it, and make clear what the argument rests on - which parts you established
     and which are assumptions you did not check. Someone else should be able to follow it and
     judge whether it holds. The form of the argument is yours to choose.
   - If you cannot tell either way, say so, and say what would settle it.

3. What will you do on the next attempt?
```

Formalization questions, AWM runs of the later harness.

```
4. Before you answer this one, bring your formal system up to date. It is the thing you are
   building across attempts; the proof is downstream of it. Work in this order, in your
   working directory:

   a. Objects. What things are on the board, and what slots does each thing carry? For every
      object you keep, you should be able to name a pair of positions that stop being
      distinguishable if you drop it.
   b. Rules. Write each rule as a function: given a position and an action, what does it
      change? Each rule needs a domain you can actually execute.
   c. Simplify. Drop slots no rule reads. Merge rules that always decide alike. After this,
      your model's verdict on every frame you have observed must be unchanged.
   d. Formalize. Turn the objects and rules into Lean *definitions* - not theorems. Each rule
      ends up in exactly one of three states: a definition, a consequence of definitions, or
      explicitly marked "not in the model".

   If you already have a formal system from an earlier attempt, **amend it. Do not start a new
   one.** What you learned this attempt is a change to specific objects, rules or definitions;
   make those changes and leave the rest alone. A rewrite throws away the part of your work
   that was already right, and it destroys the record of what this attempt actually taught you.

5. Now prove your answer to question 2 **on that formal system**: that the level can be won,
   or that it cannot.

   - The proof must compile, and `#print axioms` on your main theorem must come back clean -
     no `sorryAx`, no `native_decide`. Report that output verbatim.
   - Say what your theorem actually quantifies over and what it assumes. If the proof needs a
     premise you did not establish, write it as an explicit hypothesis rather than burying it
     in a definition.
   - Name the rules from step 4b that your theorem leans on. If your Lean definitions carry
     something your rules do not - an extra assumption, a tighter domain, a step treated as
     atomic - say so here. A proof that is clean about the wrong system is worth nothing, and
     a model that is *stricter* than the game is the dangerous direction: "impossible in my
     model" does not transfer to the game.
   - Put the file in `lean/` and tell me its filename.
   - If you cannot complete the proof, say exactly where it breaks and what is missing. An
     honest "it does not go through, here is the gap" is worth more than a theorem that proves
     something weaker than it looks.

6. If this attempt taught you nothing that changes your formal system - no rule revised, no
   definition touched - do not invent a change. Say so instead, on its own line, in exactly
   this form, and then give the reason:

       FORMAL-UNCHANGED: <why this attempt added nothing to the model - what you tried, what
       you already knew, and what would have had to happen for the model to change>
```

```
   "No change, and here is why" is a real result. A change made only to look busy is not.
```

Formalization question, AWM runs of the earlier harness.

```
4. Whichever way your answer to question 2 goes, prove it in Lean now, using the formal
   apparatus you were given in this working directory: that the level can be won, or that it
   cannot.

   - The proof must compile, and `#print axioms` on your main theorem must come back clean -
     no `sorryAx`. Report that output verbatim.
   - Say what your theorem actually quantifies over and what it assumes. If the proof needs a
     premise you did not establish, write it as an explicit hypothesis rather than burying it
     in a definition.
   - Put the file in `lean/` inside your working directory and tell me its filename.
   - If you cannot complete the proof, say exactly where it breaks and what is missing. An
     honest "it does not go through, here is the gap" is worth more than a theorem that proves
     something weaker than it looks.
```

Verdict lines, AWM runs.

```
End your reply with exactly these two lines, on lines of their own, with nothing after them:

VERDICT: winnable | impossible | impossible_in_model_proved | impossible_in_model | undecided
CONFIDENCE: <an integer from 0 to 100>

The two "in_model" words are for when the level cannot be won under the rules you yourself
measured, and the step from "those rules" to "the real game" is an assumption you are stating
rather than something you established. Say which assumption, in your answer to question 2.
Use `impossible_in_model_proved` only if you are also handing over a Lean proof of it that
compiles and whose `#print axioms` is clean - name the file. Otherwise use `impossible_in_model`.
Your proof will be compiled and its axioms checked, so do not claim `_proved` on a file that
does not go through; `impossible_in_model` with an honest account of the gap is worth more.

Answer in writing only. Do not send any more game actions.
```

Verdict lines, bare runs.

```
End your reply with exactly these two lines, on lines of their own, with nothing after them:

VERDICT: winnable | impossible | undecided
CONFIDENCE: <an integer from 0 to 100>

Answer in writing only. Do not send any more game actions.
```

Retry message.

```
You have another attempt on that level.

`POST /reset` restarts the level you were on; the levels you already beat stay beaten, and the
level's own counters start over. Play it again, as far as you can. When the attempt ends, stop
and report as before.
```

Final interview, questions.

```
That attempt failed.

Answer the following in writing. Give each answer the length it needs; do not pad.

1. What happened on the level you were on, and what do you think made it fail?
   Explain in plain prose, as if to someone who did not watch you play.

2. Do you still see a way to beat this level?
   - If you do: give the line of play, and say what you checked it against.
   - If you no longer do: say why not. State precisely what you are claiming, give the
     argument for it, and make clear what the argument rests on - which parts you established
     and which are assumptions you did not check. Someone else should be able to follow it and
     judge whether it holds. The form of the argument is yours to choose.
   - If you cannot tell either way, say so, and say what would settle it.

3. Which rule did you newly work out across these attempts, and from which observation?

4. Which rule are you still unsure about, and unsure in what way?
```

B.17 Level 5: the fresh-arena batch. The level-5 runs of §4.2 were made on 2026-10-18 with new case identifiers, each building a fresh arena, under the original wording and the same harness,

engine and prompts as before: bare Claude, bare Codex and the AWM framework three times each, plus three model runs, eight attempts each. Every Codex attempt was checked in its session log to have run `gpt-6-astra` with a writable workspace. One Codex run stopped at attempt 7 when the provider reported the model at capacity; its first six verdicts were `undecided`, it is not counted, and it was replaced by a spare run. The three model runs reported impossibility within their models from attempt 1 and kept it in every attempt; each submitted Lean that compiles with clean axioms, with its bound taken from a search outside Lean. The batch's three framework runs reported impossibility within their models: `l5r-case-3d9406` and `l5r-case-6ec36b` by an exhaustive search of their models, each putting the least winning cap at 36, the engine's value, and `l5r-case-91a4ad` by a Lean proof, over a kernel-checked table of the 2,929 states its model can reach within the cap, that no action list of its model wins within 35. Because two of these bounds come from a search outside Lean, the framework rows of Table 3 are taken from the later batch of §B.18, whose method asks for every bound to be proved in Lean. This batch differs from the earlier level-5 runs above in one respect: the earlier bare Claude runs stayed undecided over eight attempts (§B.9), while all three fresh ones asserted impossibility at attempts 4–6. Read together, the level-5 bare runs show that a bare agent may or may not assert impossibility within eight attempts, and that when it does, the argument is a hand count that a reader cannot check.

B.18 Level 5: framework runs with closed proofs. Two further framework batches were run on 2026-10-19 and 2026-10-20, three runs each, with new case identifiers, fresh arenas and asks identical to those of the fresh-arena batch. The only difference is a section added to the framework's method: the bound is to be proved in Lean over the model itself, the theorem quantifying over every action list and the tables it rests on checked by the kernel; the only hypotheses allowed restrict which actions are legal; and a second theorem shows that loosening one limit of the model by a stated amount admits a win. Both versions of the section, and the criteria for calling a proof closed, were preregistered before launch. The first version also said that a run unable to close its proof should say which step stays outside Lean. Under it one run of three closed its bound (`l5c-case-1d26a7`: no action list of its model wins within 35, with no hypotheses, and one more unit admits a win, as on the engine); the other two proved only that one fixed route fails. Two earlier launches, with drafts of this section, are not counted: one was stopped in its first attempt so that the wording could be confirmed, and one was voided after the memory guard repeatedly killed a run's Lean compilations, after which a note on memory was added to the section; one of the voided runs had completed its eight attempts without closing its bound. The second version replaced the sentence on unclosed proofs by advice to change the model's representation when its table is too large to check, and to treat a failure to close as a reason to revise the model. All three of its runs closed their bound; they are the framework rows of Table 3.

The three closed theorems were rebuilt from source in a fresh directory and use only `propext`, `Quot.sound` and, in one run, `Classical.choice`. In `l5d-case-4f4628` the theorem also takes as a hypothesis the model's own budget law, an inequality over the numbers of actions, completed spells and refills in the run; we count it as a rule of the model, not as a computed fact. Replayed from each run's server log, every recorded level-5 transition, 280 to 319 per run, is reproduced by the run's final Lean model and satisfies its certificate. Checked against the 17,194,487 engine transitions reachable at cap 35, no model contains the engine: among other gaps, each omits the cells of an opened barrier, which a block may enter on the engine. On the transitions between states it covers, the engine respects the certificate of `4f4628` on every edge, that of `l5d-case-284967` on all but 8 grow edges, each one unit cheaper on the engine, and that of `l5d-case-977714` on all but 111 edges, the engine's shot from below the first barrier, which its model prices 12 to 14 units higher. Evaluating each model's Lean definitions on the engine's cheapest route locates the excess. In `284967` the route runs out of budget exactly on the winning move, and a variant that does not charge that move wins one unit above the cap, as the engine does. In `977714` the route never wins until the shot distance is widened, after which it wins at 37, and the last unit is again the winning move. During one run's second attempt the memory guard stopped three of its compilations; its final theorem was compiled afterwards.

B.19 Level 5: the repair round. The repair round followed the level-4 hint round (§B.14) and was preregistered before launch. Each repair continues the run's own session, with the service restarted from level 1. The hint names one rule by the run's own label and says only that it does not match the game, nothing about how it differs and no costs or routes; the run is asked to re-close its proof as the

method requires, to keep reproducing every recorded transition, and to report its least budget against the budget it measured. The rules named were those located in §B.18: for `l5d-case-977714` its rule for when a shot opens a barrier, then its budget rule; for `l5d-case-284967` its budget rule. After the first hint `977714` ran five firing experiments, found that a shot from 12 or 16 cells below the first barrier opens it and that facing matters, rewrote the rule as a beam along the facing direction with no distance limit, added the facing to its state, and its least winning cap fell from 47 to 37. After the second it found from its recorded game-overs and one experiment that the budget is checked after every action, on the net count including that action's refill; it states that whether a winning move counts on the step that exhausts the budget is untested, and it chose the looser reading, which is the engine's, so that its least cap became 36. `284967` refitted its budget rule to every bar reading and every game-over of its eleven level-5 segments, 427 transitions, confirmed the fit with two experiments, and now uses the engine's cap and costs; it also built a model at the level of single clicks, so that grouping clicks into spells is no longer an assumption of its proof. Its least cap stays at 37; a probe that applies a winning move even when it cannot be paid makes its cheapest route win at 36 and not at 35, so the remaining unit is the charge on the winning move alone. We rebuilt both final proofs from source: `977714` proves with no hypotheses that no action list wins within 35 and gives a winning list at 36; `284967` proves that no legal action list wins below 37, its only hypothesis being action legality, and replays its 427 recorded transitions in the kernel. Every level-5 transition recorded by either run, 453 and 427 including those of the repair session, is reproduced by its final model and satisfies its certificate. Neither model contains the engine: both still omit cells of an opened barrier, and `977714`, having removed grow placements it had never observed, now forbids some that the engine allows, where its certificate is off by one unit; none of them lies on the winning route.

B.20 Repair after the eight attempts: the self-audit round and the bare comparison. The repairs of §4.3 follow the eight attempts of each run and use one generic signal: the engine has a cheaper winning route than the run's analysis allows, because at least one assumed rule forbids, or charges more for, a move the engine allows; no rule, cost or route is named. The two AWM runs and the four bare runs with wrong bounds received it in the same round; the text of the message differs slightly between the arms, as follows.

AWM (`m15fw-case-9ec8d4`, level 4; `l5d-case-977714`, level 5). Each run resumes its own session after its eight attempts and final interview, at the point right before the first hint it received in the hint rounds of §B.14 and §B.19, with its workspace restored to that point, so no hint is in its context. The signal is followed by the instruction to apply a self-audit section added to the method: relax each rule of the model only as far as every recorded transition allows, take the relaxations that lower the least winning cost as load-bearing, test each in the game at the first transition where the relaxed and the original model disagree, and revise, replay and prove again. Attempts are counted from the run's own resets; the run may play any level through the service, and we count its actions and resets. The level-5 run reaches 37, the engine's 36 plus its charge on the winning move, which it marks untestable, in four attempts on level 5 (104 actions, one game over); its rebuilt proof states that no action list wins within 35. The level-4 run reaches 19 in six attempts on level 4 (120 actions, two game overs); its rebuilt proof states that no action list wins within 18 under its explicit premises, with a witness at 19. The self-audit was developed on these two runs: the level-5 repair used its third version; on level 4 the third version missed the faulty case and the fourth was stopped, and the fifth, which adds that a case may be called unreachable only after a search of the model, found it. Each version was run from the same restored point.

Bare (`bast2-case-44ec53`, `bast2-case-4d3b31`, level 4; `l5r-case-339b0a`, `l5r-case-202da3`, level 5). Each run resumes its own session after its final interview, with the service restarted from level 1. The signal adds that it says nothing about whether the level can be won within its budget, and a plain-language paragraph with the same main logic as the self-audit: a rule stricter than the game makes the least cost too high, so the rules the least cost depends on should be tested in the game, especially in situations never played. At most four further attempts are allowed; the driver resets the level at the start of each and an attempt ends at a game over; after each the run reports its least budget and verdict. No run reset the level itself. Both level-4 runs report 19 from their first further attempt on (one in energy units of two per move, 38 against a measured 36), and both end `undecided`; neither bound is proved. On level 5 the runs end at 42 against a measured effective budget of about 40 (`undecided`) and at 68 against 64 in rows of the budget bar (`impossible`); neither states the engine's optimum in its own units. Read with a conversion by

hand, two actions beyond the budget of which the second enters the goal, both correspond to 37, the engine's 36 plus the winning move; we do not count them, because the conversion is ours and the runs' own budgets are approximate. Replayed on the engine, every bare run played at least one route of the engine's least cost during the round and stopped one uncharged move short of the goal.

The message in the two arms. Both arms get the same signal. The AWM message then points to the self-audit section of its method; the bare message states the same main logic in plain language and adds that the signal says nothing about whether the level can be won within its budget. The AWM rounds were run before the bare ones, and their attempts were counted from the run's own resets rather than started by the driver, which is how one of them reached six attempts, two more than the round allows. An AWM round with driver-started attempts is planned.

C Frozen rules on unseen states, and Lean against Python

Both tests were preregistered, with criteria frozen before running, and their main readings were audited independently; deviations are listed at the end of each paragraph. Test data come from the official game sources run locally; replaying the recorded winning runs on these sources reproduces every recorded frame (lp85 200/200, ft09 168/168), and no file in the benchmark repository is named after, or generated for, the modified levels.

Frozen rules on unseen states. For lp85 and ft09 we compare the concepts written during play (before abstraction: 19 for lp85, 9 for ft09) with the abstracted rules (6 and 7), both frozen at the benchmark commit. Test transitions come from random button presses (seeds 0–9) and keep only states whose screen never occurs in the recorded run. Three sets per game: new trajectories on the original levels (T1), and two modified levels. For lp85 the modification swaps tiles on level 6 (same mechanics, new layout). For ft09 it changes level 2's hidden click kernel from the identity to a plus shape, which the screen does not show. A rule set accepts a true transition if no rule judges it false; it rejects a wrong one, made by the benchmark's own five corruption operators, if some rule judges it false. Table 8 gives the readings; confidence intervals resample whole episodes. The modified levels come in pairs. In lp85 tile swap U, one goal tile is moved so that level 6 cannot be won; D is U with one more swap, which makes it winnable again. In ft09 kernel change FU, the plus-shaped kernel makes level 2 unwinnable; FD adds a one-pixel change to that level's template, which makes it winnable again. All four were checked by exhaustive search on the modified engine.

Table 8: Frozen rules on unseen transitions: before abstraction / after (%). U, D: the unwinnable and winnable tile swaps of lp85 level 6; FU, FD: the unwinnable and winnable kernel changes of ft09 level 2 (see text). "Wrong rejected" is the range over the four corruption operators for which both rule sets can read both frames; the fifth, which edits changing cells, is reported in the text.

Set	n	True accepted	Wrong rejected	Next frame predicted
lp85 T1 (new trajectories)	2,073	100 / 100	99.5–100 / 99.5–100	99.95 / 99.95
lp85 tile swap U	2,080	100 / 100	100 / 100	100 / 100
lp85 tile swap D	2,080	100 / 100	100 / 100	100 / 100
ft09 T1 (new trajectories)	2,026	99.0 / 100	99.0–100 / 100	—
ft09 kernel change FU	2,015	0 / 0	—	—
ft09 kernel change FD	2,016	0 / 0	—	—

No set meets the preregistered criterion for a benefit of abstraction (a gain of at least 5 points in every comparable cell, with intervals excluding zero). The tests sit near their ceiling: before abstraction the concepts already accept 99–100% of true transitions and reject 99–100% of wrong ones on the four comparable operators, so a 5-point gain is not possible. We therefore claim neither a benefit nor its absence. Three readings stand out. On ft09 T1 the abstracted rules accept 1.0 point more true transitions (95% CI $[0.1, 2.2]$). On the lp85 modified layouts, the corruption that edits changing cells is rejected by 1 of 2,080 cases before abstraction and by all after, but the rejection after abstraction comes from the object parser refusing the corrupted frame, not from a rule. On the ft09 kernel change both rule sets reject every true transition, all through the rule that nothing else changes: each click now changes 110 or 146 cells instead of 38 on the original level. The models fail loudly on dynamics they could not have observed, rather than accepting them silently. Next-frame prediction for lp85

compares the abstracted permutation with the union of the earlier concepts' predictions on tile cells; a first version also required the earlier concepts to predict the step bar, which the abstracted rule does not predict either, and was discarded as unequal.

Lean against Python on the same model. On the framework run's level-5 model and a second one, we injected errors into three places and asked whether each route still reported the bound: the search or certificate generator (8 mutants), the model definition (7), and the theorem statement (7, plus 2 constructions with contradictory premises or an unreachable goal). Mutants were frozen before the runs.

- *Search or generator errors.* Lean's kernel rejected every invalid certificate. With the expected bound asserted, as preregistered, the Python route also accepted none. Two readings were added afterwards: a Python search that only prints its result accepted 5 wrong bounds, and a Python checker of the same certificate conditions as Lean agreed with Lean on all 8. The difference is having a checker, not the language.
- *Model errors.* Both routes accepted the same wrong models; mutations that changed the model but not the bound went unnoticed by both.
- *Statement errors.* With only a lower-bound theorem, both routes accepted vacuous bounds (a goal that can never be reached, contradictory premises). Adding a witness that the goal is reachable made both reject them.
- *Premises.* In three of the agents' Lean developments the key conclusion rests on premises the kernel does not check. In one, the premise is the output of the agent's Python search, a minimum cost of 46; rerunning that search gives 43, and the premise is false. Lean makes such premises visible in the theorem statement; it does not discharge them.

In these tests, Lean's distinct contribution is auditability: the trusted part shrinks to the model definition and the statement, and the premises are written into the theorem. Reliability about the environment then comes from checking that model against it, and the theorem says exactly what to check. Deviations: one defect in our own certificate generator, a sign error, was found and fixed during the runs, and all mutants were rerun with the fixed version, keeping the original readings; the run script was modified after the mutants were frozen and its frozen copy was not kept; and the Python model is our hand translation of the Lean model, checked against it transition by transition, so the two routes are not independent implementations.

D Figures moved from the main text

The figures below appeared in the main text of an earlier version and are kept here unchanged. Figure 6 is the complete model of `lp85`, whose panels (d) and (e) are part (b) and (c) of Figure 3; Figure 7 gives all three panels of level 4, whose panels (a) and (c) are Figure 4, and whose panel (b) shows the verdict after every attempt of every run summarized in Table 3.

E Tables moved from the main text

Per-game official scores (summarized in Table 1) and per-game certified actions.

References

Diego Aineto and Enrico Scala. Action model learning with guarantees. In *Proceedings of the 21st International Conference on Principles of Knowledge Representation and Reasoning (KR)*, pp. 801–811, 2024. doi: 10.24963/kr.2024/75.

Edmund M. Clarke, Orna Grumberg, and David E. Long. Model checking and abstraction. *ACM Transactions on Programming Languages and Systems*, 16(5):1512–1542, 1994. doi: 10.1145/186025.186051.

Leslie Pack Kaelbling, Michael L. Littman, and Anthony R. Cassandra. Planning and acting in partially observable stochastic domains. *Artificial Intelligence*, 101(1–2):99–134, 1998. doi: 10.1016/S0004-3702(98)00023-X.

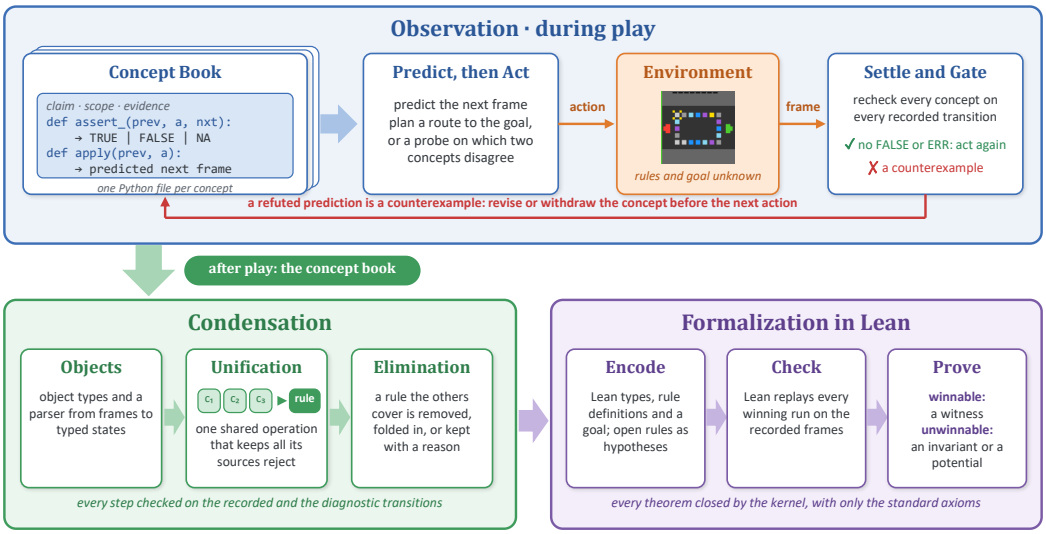


Figure 1: The AWM pipeline. Observation keeps a book of executable concepts that is re-checked on every recorded transition after each batch of actions, before the next one is allowed, and a result that refutes a prediction is a counterexample that has the concept revised or withdrawn before the agent acts again; abstraction extracts object types and rewrites the concepts through unification and elimination; formalization encodes the result in Lean, checks the encoding by replaying every winning run, and proves reachability and impossibility within the model.

(a) One concept of lp85, as the agent wrote it during play

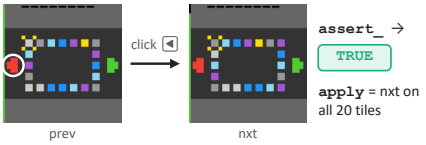
```
"""When the clicked arrow points against its belt's order,
   every tile color moves one place along the order."""
LEVEL    = "L1"                    # where it was first seen
SCOPE    = "clicks on a belt arrow; level change: NA"
EVIDENCE = ["4c0c8a2f…", "02bfcf1d…", …]  # 8 cited steps

def assert_(prev, action, nxt, ctx):       # checker
    order = belt(prev, action)
    if order is None: return "NA"
    a = [color(prev, p) for p in order]
    b = [color(nxt, p) for p in order]
    return "TRUE" if b == a[1:] + a[:1] else "FALSE"

def apply(prev, action, ctx):                # predictor
    order = belt(prev, action)
    if order is None: return None
    a = [color(prev, p) for p in order]
    return paint(order, a[1:] + a[:1])   # predicted cells
```

Abridged; docstring and scope translated, helper names shortened.

(b) On a recorded transition



1 Three verdicts, not two

A click off every belt arrow gives `NA`, not `TRUE`: "does not apply" is kept apart from "holds". A `FALSE` on any recorded step blocks the next action.

2 Not scored on its own evidence

It is scored only on the recorded steps it does not cite in `EVIDENCE`.

3 Both halves must match

`apply`'s prediction is compared with the next frame, so a checker that passes while the predictor is wrong is caught.

Figure 2: A concept. (a) The concept of `lp85` for a click on an arrow that points against its belt's order, abridged from the file the agent wrote during play. (b) On the first click of the certified run of level 1, its checker returns `TRUE` and its predictor reproduces the next frame on all 20 tiles. A concept is scored only on recorded transitions outside its own `EVIDENCE`, and a `FALSE` on any recorded transition, or an exception counted as ERR, blocks the next action (Eq. (1)).

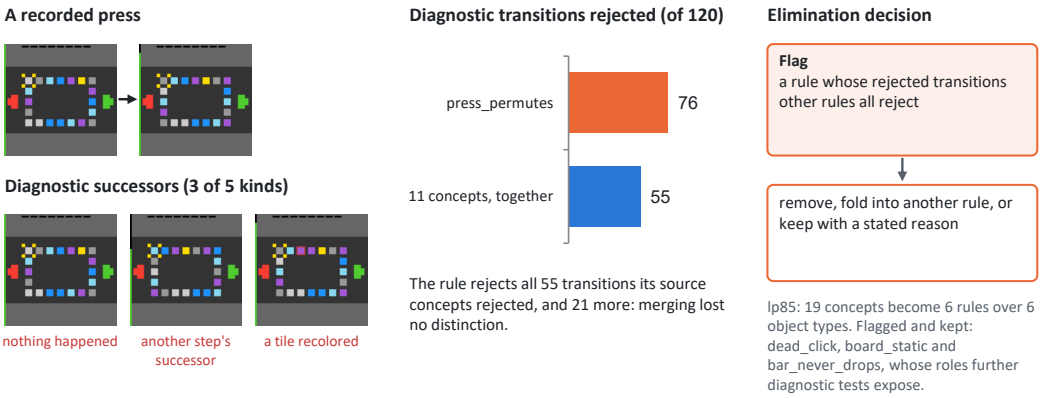


Figure 3: Diagnostic transitions, the merge check and elimination in `lp85`. Diagnostic transitions are built from recorded ones (three of the five constructions are shown on a real press). On the 120 diagnostic transitions $\mathcal{N}$ of this game, the merged rule `press_permutes` rejects all 55 that its eleven source concepts rejected together, and 21 more, so the merge lost no tested distinction; these counts are for the eleven merged concepts and their rule, while the whole book and the final six rules reject 117 and 119 (Table 2). A rule whose rejections the others cover is flagged and then removed, folded into another rule, or kept with a stated reason; the three flagged rules of `lp85`, `dead_click`, `board_static` and `bar_never_drops`, are kept because further diagnostic tests expose roles absent from this set.

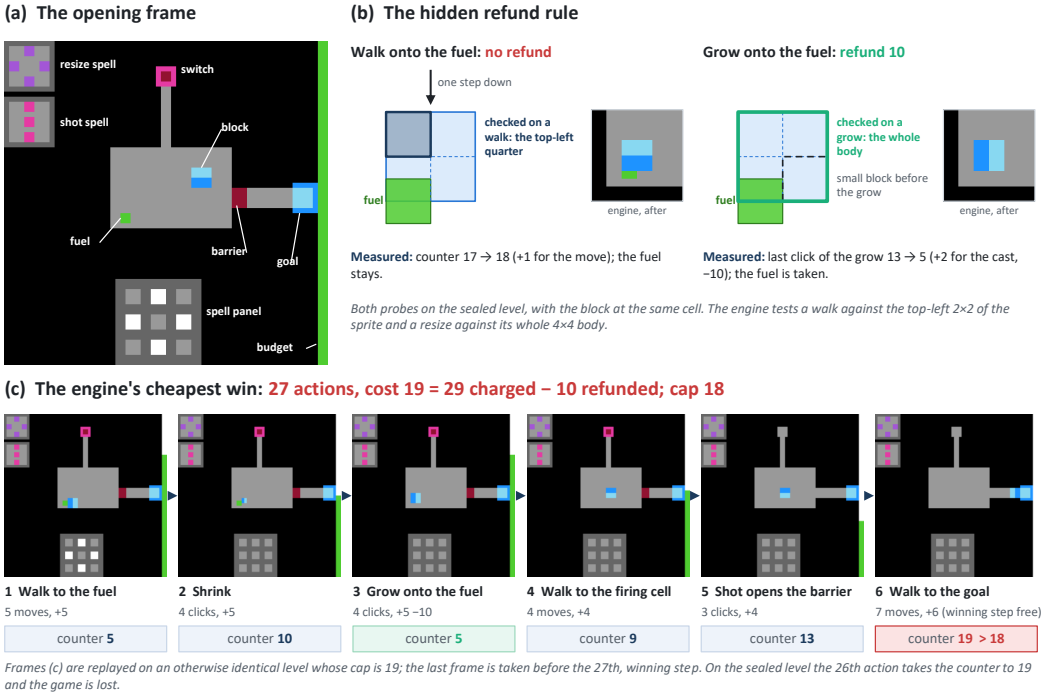
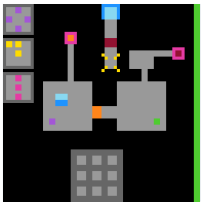


Figure 4: Sealed level 4 of `sc25`. (a) The opening frame. (b) The hidden refund rule: with the fuel under the bottom-left quarter of a large block, walking there pays nothing, because the engine checks only the top-left quarter, while growing there refunds 10; this is the rule the framework run that proved 20 misread. (c) The engine's cheapest win, 27 actions at a cost of 19 against a cap of 18, with the counter after each stage. Frames are rendered from the real engine with the official palette; those in (c) are replayed on an otherwise identical level whose cap is 19, since on the sealed level the 26th action ends the game.

Table 9: Official scores on the 25 public ARC-AGI-3 games. Baselines are the official ARC Prize results: Claude Opus 5 (High) and GPT-6 Astra (Max) under the standard harness, which provides no code execution, and GPT-6 Astra (Max) under its provider-adapter harness. AWM is scored on its certified run. baseline1 from its ARC Prize scorecard; Memento 3 as reported in (Zhao et al., 2026).

		Official score (max 100)					
Game	Lv.	Claude Opus 5	GPT-6 Astra	GPT-6 Astra (adapter)	baseline1 (Rodionov, 2026)	Memento 3 (Zhao et al., 2026)	**AWM (ours)**
ar25	8	100.0	100.0	100.0	100.0	100.0	**100.0**
bp35	9	0.0	2.2	100.0	100.0	100.0	**100.0**
cd82	6	0.6	82.9	100.0	100.0	100.0	**100.0**
cn04	6	47.6	100.0	100.0	100.0	100.0	**100.0**
dc22	6	44.8	100.0	100.0	100.0	100.0	**100.0**
ft09	6	100.0	100.0	100.0	100.0	100.0	**100.0**
g50t	7	0.0	0.0	100.0	100.0	100.0	**100.0**
ka59	7	10.4	75.0	100.0	100.0	100.0	**97.6**
lf52	10	1.8	10.9	100.0	100.0	100.0	**100.0**
lp85	8	100.0	100.0	100.0	100.0	100.0	**100.0**
ls20	7	5.4	83.5	100.0	100.0	100.0	**100.0**
m0r0	6	28.6	71.4	100.0	100.0	100.0	**100.0**
r11l	6	100.0	100.0	100.0	100.0	100.0	**100.0**
re86	8	58.3	77.8	100.0	100.0	100.0	**100.0**
s5i5	8	100.0	100.0	100.0	100.0	100.0	**99.3**
sb26	8	77.8	100.0	100.0	100.0	100.0	**100.0**
sc25	6	0.0	47.6	100.0	84.2	100.0	**100.0**
sk48	8	8.3	19.7	100.0	100.0	100.0	**100.0**
sp80	6	56.3	100.0	100.0	100.0	100.0	**100.0**
su15	9	10.4	27.4	100.0	100.0	100.0	**100.0**
tn36	7	6.0	53.6	100.0	90.1	100.0	**100.0**
tr87	6	47.6	100.0	100.0	100.0	100.0	**100.0**
tu93	9	3.6	3.1	100.0	100.0	100.0	**100.0**
vc33	7	98.8	73.3	100.0	100.0	100.0	**100.0**
wa30	9	10.6	80.0	100.0	100.0	100.0	**100.0**
Mean	183	40.7	68.3	100.0	99.0	100.0	**99.9**
Games at 100		5	10	25	23	25	**23**



sealed level 5, cap 35;
cheapest win:
42 actions,
cost 36

Algebraic world model

Objects and state

row, col | size | facing | barriers | fuel

Rules	cost
move: 4 cells if large, 2 if small	1
resize: shrink in place, grow up-left	5
shot: a beam along the facing opens the first barrier it hits	4
teleport: a fixed landing per size	4
fuel: the top-left lands on it	−10
win: the block enters the goal	

budget: checked after every action;
a winning move wins before the check

Lean proof L5Model.lean

```
theorem l5_impossible :
  ∀ as, runB 35 s0 0 as = false
theorem witness_wins :
  runB 36 s0 0 witness = true
```

a table V over 4,710 modeled states, each entry checked by the kernel

1. start: $V(w_0) = 37$
2. each step: $V(w) \le \text{cost}(w, a) + V(w')$
3. the winning move costs 1

⇒ **every win needs a budget** $\ge 36 > 35$

✓ **no hypotheses; axioms: propext, Quot.sound**

Against the engine. The model reproduces all 453 level-5 transitions its run recorded, and its least budget, 36, is the engine's. It departs from the engine only away from the winning route: it leaves out the cells of an opened barrier, and next to one wall it does not let a small block grow in place.

Figure 5: Sealed level 5 and a closed certificate for it: the model and Lean proof of the third framework run of Table 3 after the repair round (detailed in Appendix B.19). Left: the opening frame. Middle: the objects and state of its model, and its rules with their costs. Right: its Lean theorems, proved by the potential argument of §2.4 over a kernel-checked table; the main theorem has no hypotheses. The table ranges over the 4,710 states of the typed model, against about 1.35 million engine states (Table 4); the object types are what make it small enough for the kernel to check. Bottom: how the model compares with the engine.

Table 10: Certified actions per game. Human, baseline1 and Memento 3 as reported in (Zhao et al., 2026); AWM from its certified runs (Table 1).

Game	Human	baseline1	Memento 3	AWM	AWM / Human
ar25	748	271	253	253	0.34
bp35	651	532	429	374	0.57
cd82	171	93	82	70	0.41
cn04	789	201	201	261	0.33
dc22	1,228	898	643	1,037	0.84
ft09	208	85	75	80	0.38
g50t	879	363	307	288	0.33
ka59	730	362	341	421	0.58
lf52	1,339	767	894	905	0.68
lp85	388	90	95	79	0.20
ls20	776	465	473	368	0.47
m0r0	1,107	215	220	215	0.19
r11l	233	97	75	81	0.35
re86	1,255	559	625	547	0.44
s5i5	638	283	261	461	0.72
sb26	213	140	124	124	0.58
sc25	350	619	173	146	0.42
sk48	1,070	419	407	263	0.25
sp80	518	193	144	138	0.27
su15	361	117	104	90	0.25
tn36	317	292	133	125	0.39
tr87	414	147	148	119	0.29
tu93	462	201	208	187	0.40
vc33	447	293	204	167	0.37
wa30	1,843	645	899	649	0.35
Total	17,135	8,347	7,518	**7,448**	0.43

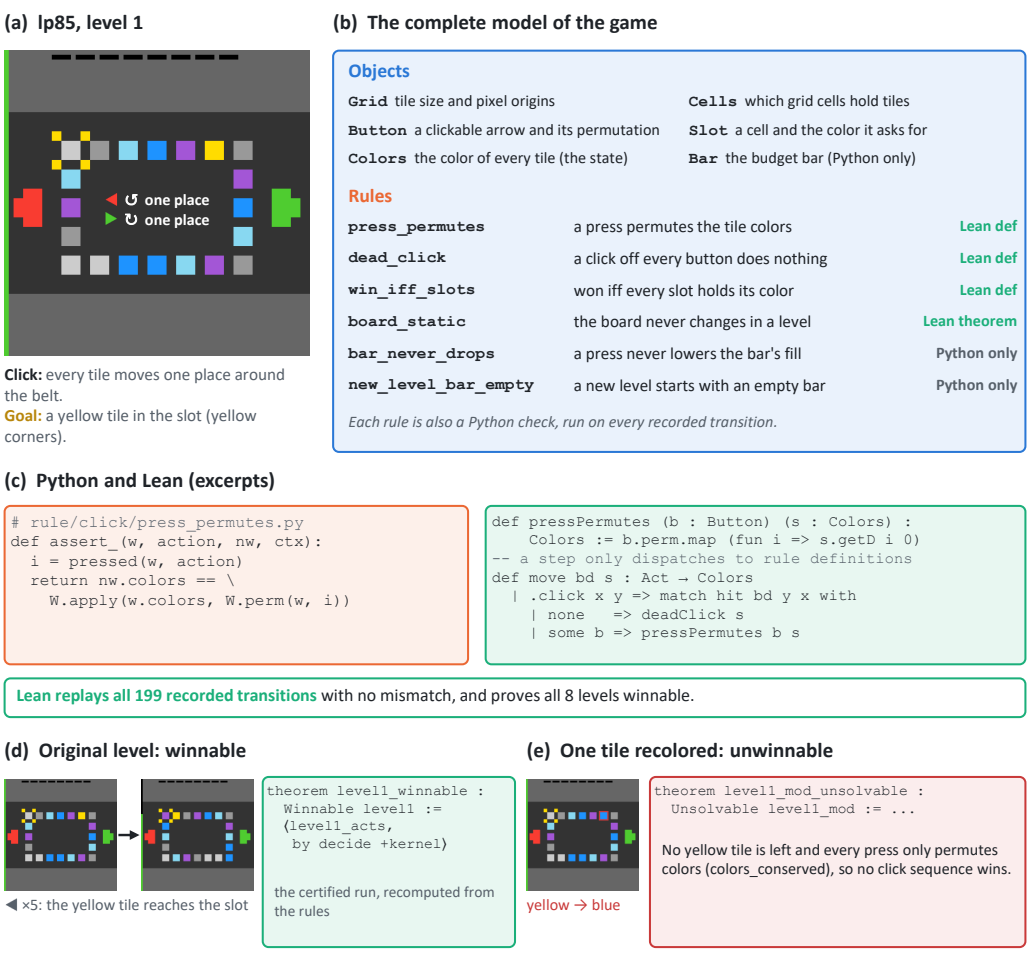


Figure 6: The complete algebraic world model of an ARC-AGI-3 game. (a) Level 1 of `lp85`: 20 colored tiles form a closed belt, and each click on an arrow moves every tile one place around the belt, counterclockwise for the left arrow and clockwise for the right; the level is won when a yellow tile stands in the slot marked by yellow corners. (b) All objects and rules of the model, with the form each takes in Lean; the two rules about the budget bar stay in Python. (c) Excerpts: one rule as its Python check, the same rule as a Lean definition, and the Lean step, which only dispatches to rule definitions. Lean replays every recorded transition of the game with no mismatch. (d) On the original level, Lean checks a winning witness, the certified run. (e) With the only yellow tile recolored, the conservation law of `press_permutes` shows that no click sequence wins. Both statements are theorems about the model.

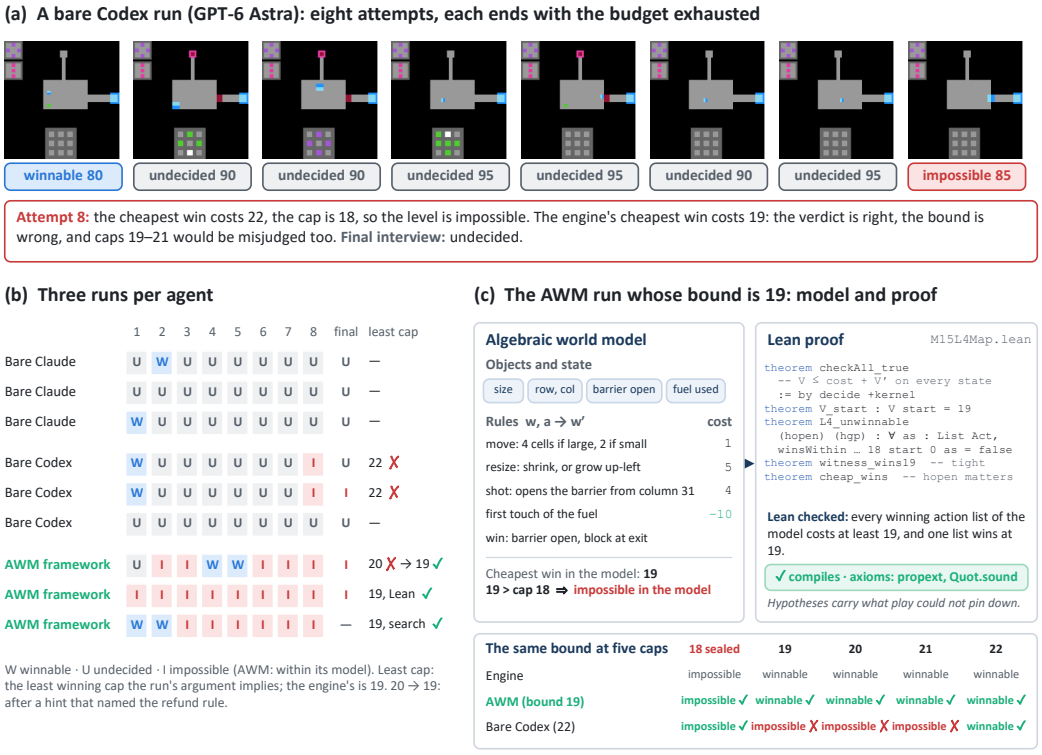


Figure 7: Level 4, attempt by attempt. (a) A bare Codex run: every attempt ends with the budget exhausted; at attempt 8 it asserts impossibility from a cheapest win of 22, where the engine's is 19 (both net), and it withdraws the verdict in the final interview. (b) The verdict after each attempt in the three runs of each agent, and the least winning cap implied by each run's argument; the engine's is 19. The first framework run proved 20 and, after a hint that named the refund rule, 19; the third has no recorded final verdict, and its Lean check did not finish. (c) The model and proof of the AWM framework run whose bound is 19: its objects and state, its rules with their costs, and its Lean theorems, with the bound compared with the engine's at five caps. Spells are single actions in this model. Frames are replayed from the run's action log on the sealed engine.

Sergey Rodionov. Do coding agents need executable world models, simplification, and verification to solve ARC-AGI-3? *arXiv preprint arXiv:2607.15439*, 2026.

Haoyu Zhao, Zhengxu Yu, Zhiyuan He, Meng Fang, Rasul Tutunov, Haitham Bou-Ammar, Weilin Luo, and Jun Wang. Memento 3: Model-based recursive self-improvement through reflective rulebooks. *arXiv preprint arXiv:2610.11794*, 2026.